

Document Loaders in LangChain

1. Introduction

In real-world AI applications, data doesn't come from prompts only.

👉 It comes from:

- PDFs
- Text files
- Websites
- CSV files

But LLMs **cannot directly read these files**.

👉 So we need **Document Loaders**

2. AI Auto Dubbing Feature (Quick Note)

This part is just a feature explanation (not core to topic):

- AI can translate and dub content automatically
- Shows how LLMs can work with **real-world media/data**

👉 Important idea: LLMs need **external data sources**

3. Recap

You've learned:

- Runnables
- Chains
- Output Parsers

Now we move toward:

👉 **Real-world applications using external data**

4. What is RAG?

✅ **RAG = Retrieval-Augmented Generation**

💡 **Simple Meaning:**

Combine LLM with external data to give better answers

Without RAG:

LLM answers from its training only ❌

With RAG:

LLM uses:

- Documents
- Files
- Databases

👉 More accurate answers 

Example:

Question:

"Summarize my PDF"

👉 LLM needs access to that PDF → RAG

5. RAG Components

A typical RAG system has:

- **Document Loader** → Load data
 - **Text Splitter** → Break into chunks
 - **Embeddings** → Convert to vectors
 - **Vector Database** → Store data
 - **Retriever** → Find relevant info
 - **LLM** → Generate answer
-

👉 Today's focus:

★ **Document Loader (Step 1)**

6. Document Loaders in LangChain

 **What is a Document Loader?**

A **Document Loader**:

Loads data from different sources and converts it into a standard format

💡 **Output Format:**

Usually returns:

Document(page_content="text...", metadata={...})

 **Why needed?**

Because:

- LLM cannot read files directly
 - Files must be converted into text
-

7. Text Loader

What it does:

Loads **.txt** files

Example:

```
from langchain.document_loaders import TextLoader

loader = TextLoader("data.txt")
docs = loader.load()
```

Output:

- Reads full text file
 - Converts into document format
-

Use Case:

- Notes
 - Logs
 - Simple datasets
-

8. PyPDFLoader

What it does:

Loads **PDF** files

Example:

```
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("file.pdf")
docs = loader.load()
```

Behavior:

- Extracts text from PDF
 - Splits into pages
-

Output:

Each page = one document

Use Case:

- Research papers
 - Books
 - Reports
-

9. Limitations

Common Problems:

1. **Messy formatting**
 - PDFs may have broken text
 2. **Images not readable**
 - Needs OCR (extra tools)
 3. **Tables not structured properly**
 4. **Encoding issues**
-

Important:

Document loaders are **not perfect**

10. Directory Loader

What it does:

Loads **multiple files from a folder**

Example:

```
from langchain.document_loaders import DirectoryLoader

loader = DirectoryLoader("data/")
docs = loader.load()
```

Behavior:

- Reads all files in directory
 - Uses appropriate loader internally
-

Use Case:

- Large datasets
 - Multiple documents
-

11. Load vs Lazy Load

load()

- Loads everything at once
-

- Faster for small data

✅ **lazy_load()**

- Loads data step-by-step
- Memory efficient

💡 **Analogy:**

- load() = download full movie 🎬
- lazy_load() = stream video 📺

✅ **When to use:**

Method	Use Case
load()	Small files
lazy_load()	Large datasets

12. WebBaseLoader

✅ **What it does:**

Loads data from **websites**

Example:

```
from langchain.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://example.com")
docs = loader.load()
```

💡 **Behavior:**

- Scrapes webpage
- Extracts text content

✅ **Use Case:**

- Blogs
- Articles
- Documentation

13. CSVLoader

✅ **What it does:**

Loads **CSV (tabular data)**

Example:

```
from langchain.document_loaders import CSVLoader

loader = CSVLoader(file_path="data.csv")
docs = loader.load()
```

Behavior:

- Each row → one document

Example CSV:

```
name,age
Ali,22
Sara,21
```

Output:

```
"Ali, 22"
"Sara, 21"
```

Use Case:

- Structured data
- Databases export

14. Other Document Loaders

LangChain supports many loaders:

Examples:

- JSONLoader
- UnstructuredFileLoader
- NotionLoader
- GoogleDocsLoader

Meaning:

You can load data from **almost anywhere**

15. Custom Document Loader

What it is:

You can create your **own loader**

When needed:

- Unsupported file format
 - Special data source
 - Custom preprocessing
-

Example:

```
class CustomLoader:
    def load(self):
        return [Document(page_content="custom data")]
```

Key Idea:

 You control how data is loaded

Big Picture (VERY IMPORTANT)

Full RAG Flow:

```
File → Loader → Text → Split → Embeddings → Vector DB → Retrieval → LLM
```

 Document Loader is the **first step**

Without it → nothing works

Important Points to Remember

1. LLMs cannot read files directly
 2. Document Loaders convert files → text
 3. Different loaders for different formats
 4. Use `lazy_load()` for large data
 5. Data quality affects final output
-

Final Summary

- **Document Loader = tool to load external data**
 - Used in **RAG systems**
 - Types:
 - TextLoader → text files
 - PyPDFLoader → PDFs
 - DirectoryLoader → folders
 - WebBaseLoader → websites
 - CSVLoader → tabular data
-

- Supports:
 - Multiple formats
 - Custom loaders
-